BuySideFlow: A Time-Anchored Benchmark for Query-and-Analysis Agents in Chinese Buy-Side Investment Research

Anonymous Author(s)
Affiliation
Address
email

Abstract

Buy-side investment research is not just "write SQL and run it. A workflow can execute successfully and still be wrong: it may pull a filing that was not yet public at the research date, rely on a classification later revised, or apply today's security list to a date when it looked different. We introduce BUYSIDEFLOW, a time-anchored benchmark for query-and-analysis agents—concretely, executable SQL and SQL–Python workflows—in Chinese buy-side research. The benchmark contains 404 tasks across stock, fund, bond, and macro analysis, evaluated on a fixed snapshot of a licensed 108-table financial database that includes daily quotes, financial statements, fund holdings, bond valuations, index constituents, credit ratings, and macro indicators. 312 tasks are SQL-centered; 92 require Python for downstream computation or artifact generation. Submissions run under strict output contracts, with static guards blocking runtime date functions, advisory point-in-time diagnostic flags, and per-run provenance hashed against a fixed snapshot. Across seven controlled LLM agent runs covering general production-grade models and a coding-specialized variant, the strongest run solves 55 of 404 tasks; on the Python-mediated subset, the best runs reach only 8 of 92. Most failures are not crashes—they are workflows that run, return an artifact, and disagree with the reference, often after missing a classification vintage or a disclosure-date guard. Reliable temporal correctness remains out of reach for current SQL and SQL–Python agents.

1 Introduction

Buy-side investment research is not defined by a database query alone. An analyst asking for "current" index constituents, the "latest" financial statement, or a valid fund manager assignment is usually asking under a research date: the information set available when a decision, screen, or backtest would have been formed. The same SQL can execute and still be wrong if it uses a later disclosure, a backfilled classification, or a security universe revised after the intended observation date. Correctness in this setting is therefore not exhausted by semantic parsing or successful execution; the retrieved rows must also be admissible at the point in time implied by the task.

Text-to-SQL benchmarks such as Spider, BIRD, and Spider 2.0 have made database evaluation progressively more realistic, from cross-domain schema generalization to enterprise-style workflow completion [Yu et al., 2018, Li et al., 2023, Lei et al., 2025]. Chinese text-to-SQL benchmarks such as CSpider and DuSQL add schema-linking and value-grounding difficulty when questions, schemas, and database values no longer share the same surface form [Min et al., 2019, Wang et al., 2020]. Financial resources such as FinSQL and FINCH further introduce wide schemas, abbreviated fields,

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

and financial terminology [Zhang et al., 2024, Singh et al., 2025]. These settings are important, but they do not make historical admissibility a first-class correctness condition.

Financial QA and agent benchmarks cover a different part of the pipeline. FinQA-style tasks evaluate reasoning after evidence has been provided [Chen et al., 2021, Zhu et al., 2021, Chen et al., 2022, Reddy et al., 2024]. Recent financial-agent benchmarks involve tool use, public filings, browsing, and judge-based scoring [Bigeard et al., 2025, Zeng et al., 2025]. A buy-side database workflow begins earlier: it must construct the admissible universe, retrieve records from a relational financial database, apply dated business definitions, and produce a downstream research artifact. That step is where look-ahead bias often enters.

The core constraint in BUYSIDEFLOW is time anchoring. A financial statement should be available only after its disclosure date, not merely because its report period precedes the research date. An index constituent should be included only if its effective interval covers the observation date. A fund, bond, stock, classification, rating, or manager assignment should be filtered by its valid state rather than by the database's final state. Violating these rules produces the operational form of look-ahead bias in empirical finance, where information unavailable at portfolio formation contaminates the evaluation [Banz and Breen, 1986, Bailey et al., 2016, Brown et al., 1992, Lopez de Prado, 2018]. SQL–Python workflows make the failure harder to detect because leakage can hide inside joins, runtime date functions, or post-processing code.

BUYSIDEFLOW is a time-anchored benchmark for query-and-analysis agents in Chinese buy-side research. It contains 404 tasks spanning stock, fund, bond, and a smaller exploratory macro slice. Tasks are evaluated against a licensed financial database snapshot that exposes 108 documented tables to the agent; the current reference programs exercise 78 of these tables. The tasks are organized around research operations rather than query templates: point-in-time universe construction, event-centered analysis, cross-asset comparison, and deliverable generation. Semantically, 312 tasks are SQL-centered, while 92 require Python-mediated analysis or artifact generation after SQL retrieval. Each task is tied to an explicit, structured, or derivable temporal anchor.

BUYSIDEFLOW evaluates agents as executable research systems. A submission must ground the task in the available schema, produce valid SQL, use Python when downstream computation is required, and satisfy a task-specific output contract. An artifact that runs and returns the right shape can still be wrong if it relied on rows that were not observable at the research date. The public release is therefore an audit package rather than a redistribution of the licensed database: it exposes task text, metadata, evaluation code, audit records, and provenance hashes, while full execution requires controlled access to the fixed database snapshot or an official evaluation service.

Across seven controlled LLM agent runs, all models solve only a small fraction of the benchmark. Performance remains weak on both SQL-centered tasks and Python-mediated workflows. Tool-use traces show that failures are not mainly caused by missing execution attempts: agents usually inspect schema and run candidate code, but often fail to converge to the correct table choice, temporal guard, financial definition, or output format. Point-in-time diagnostics show recurring failures around classification vintages and disclosure-date guards.

The paper makes three contributions. First, we introduce a benchmark for Chinese buy-side SQL and SQL–Python agents in which research tasks are evaluated as executable workflows over a licensed 108-table financial database snapshot. Second, we define a time-auditable evaluation protocol in which executable references, strict output contracts, and leakage diagnostics test whether a workflow was valid at the stated research date. Third, we provide a license-aware reproducibility design with public audit artifacts, controlled execution access, and provenance records for the non-public evaluation state.

2 Related Work

Table 1 summarizes the closest benchmark families. We focus on whether evaluation is executable, whether the schema resembles an operational database, whether Chinese financial grounding is required, and whether point-in-time admissibility is audited.

Database and executable-workflow benchmarks. Spider, BIRD, and Spider 2.0 established progressively more realistic text-to-SQL evaluation, from cross-domain schema generalization to

Benchmark	Domain	Data	Task form	Evaluation	Zh	PIT
Spider [Yu et al., 2018]	General DB	Public	Single SQL	Match-based	–	No
BIRD [Li et al., 2023]	General DB	Public	Single SQL	Execution	–	No
Spider 2.0 [Lei et al., 2025]	Enterprise DB	Public	SQL workflow	Execution	–	No
BIRD-Interact [Huo et al., 2026]	General DB	Public	Interactive SQL	Execution	–	No
BEAVER [Chen et al., 2024]	Enterprise DB	Private	Single SQL	Execution	–	No
CSpider [Min et al., 2019]	General DB	Public	Single SQL	Match-based	Yes	No
DuSQL [Wang et al., 2020]	General DB	Public	Single SQL	Execution	Yes	No
FinSQL [Zhang et al., 2024]	Financial DB	Limited	Single SQL	Execution	Yes	Implicit
Finance Agent Bench. [Bigeard et al., 2025]	Financial agents	Public	Tool use	Judge-based	–	Implicit
FinGAIA [Zeng et al., 2025]	Financial agents	Partial	Tool use	Judge-based	Yes	Implicit
DS-1000 [Lai et al., 2023]	Data science	Public	Python	Execution	–	N/A
InfiAgent-DABench [Hu et al., 2024]	Tabular analysis	Public	SQL/Python	Execution	–	N/A
BUYSIDEFLOW	Buy-side DB	Licensed	SQL + SQL–Python	Execution + audit	Yes	Audited

Table 1: Comparison with related benchmark families. PIT denotes point-in-time admissibility. "Audited" means that temporal admissibility is checked by the evaluation protocol rather than left as prompt context.

enterprise-style workflow completion [Yu et al., 2018, Li et al., 2023, Lei et al., 2025]. SParC, CoSQL, and BIRD-Interact add conversational or interactive database use [Yu et al., 2019b,a, Huo et al., 2026], while BEAVER and BenchPress emphasize the curation and documentation burden of private or enterprise databases [Chen et al., 2024, Wenz et al., 2026]. Execution-based analysis benchmarks such as DS-1000, InfiAgent-DABench, and Text2Analysis evaluate code over data-science libraries, tables, or files [Lai et al., 2023, Hu et al., 2024, He et al., 2024]. These benchmarks make execution and workflow context central, but they generally score against the database or files available at evaluation time. They do not require the agent to prove that retrieved records were observable at a stated historical research date.

Chinese and financial benchmarks. CSpider and DuSQL show that Chinese schema linking and value grounding remain difficult when questions, schemas, and database values use different surface forms [Min et al., 2019, Wang et al., 2020]. Financial QA benchmarks such as FinQA, TAT-QA, ConvFinQA, and DocFinQA evaluate numerical reasoning over reports, hybrid table-text evidence, conversations, or long documents [Chen et al., 2021, Zhu et al., 2021, Chen et al., 2022, Reddy et al., 2024]. FinBen, Finance Agent Benchmark, and FinGAIA broaden the financial-agent setting through heterogeneous tasks, tool use, filings, browsing, and judge-based scoring [Xie et al., 2024, Bigeard et al., 2025, Zeng et al., 2025]. FinSQL and FINCH are closest on the database side because they address financial schemas and terminology [Zhang et al., 2024, Singh et al., 2025]. Their evaluation contracts, however, are not designed around disclosure dates, validity intervals, revised classifications, or future-label separation.

Point-in-time data and look-ahead bias. In quantitative finance, time leakage changes the meaning of an empirical result. A screen or backtest built with information unavailable at portfolio formation can appear statistically valid while being unusable in practice. Non-point-in-time Compustat data, survivorship bias, and repeated selection over historical outcomes are known sources of distorted empirical conclusions [Banz and Breen, 1986, Brown et al., 1992, Bailey et al., 2016]. Financial machine-learning practice therefore emphasizes disclosure timing, survivorship control, corporate-action timing, and separation between features and future labels [Lopez de Prado, 2018]. For database agents, these concerns become execution constraints: final-state entity tables, revised classifications, post-disclosure indicators, and runtime date functions are leakage channels. BuySideFlow translates this requirement into task-level time anchors, audited references, leakage guards, and fixed-snapshot provenance.

Stage	Reviewers	Main checks	Outcome
Task drafting	Research team	Realism, anonymization, initial coverage	411-task draft pool
Domain review I	11 internal practitioners	Redundancy, PIT anchors, buy-side relevance, missing workflow types	Removed near-duplicates and added under-covered workflows
Domain review II	4 external practitioners	Financial validity, metric definitions, output boundary, task wording	Tightened financial definitions and executable wording
Reference development	5 data/engineering staff	Executable SQL/Python, database consistency, output artifacts	Implemented references and materialized expected outputs
Three-pass reference review	1 senior financial data analyst + 2 senior engineers	Database execution, output contracts, reproducibility, provenance	Accepted 404 tasks

Table 2: Task construction and review workflow. Participants are reported by role and count rather than by name or institution.

3 Benchmark Design and Dataset Construction

Task format and sources. BuySideFlow evaluates executable research tasks rather than isolated SQL strings. Each instance contains a Chinese buy-side research request, optional structured inputs, an output contract, reference SQL and/or Python code, and the reference artifact used by the evaluator. Task statements are stored separately from reference programs and outputs.

Tasks were constructed from routine buy-side workflows and public investment-research materials, including strategy reviews, asset-allocation monitoring, fund due diligence, factor backtests, portfolio diagnostics, quantitative-investment discussions, financial forums, and sell-side research reports. Source documents are not released. Each task was rewritten into a standardized format, with identifying details removed or replaced by generic research settings.

Construction and review. Task construction started from a 411-task draft pool: 171 stock, 98 bond, 122 fund, and 20 macro tasks. Review changed the pool rather than simply deleting tasks. Observation-date screens and ranked lists were over-represented; event-driven analysis, factor tests, rebalancing, attribution, and leakage-sensitive cases were under-covered. Near-duplicates were removed, selected tasks were rewritten, and targeted examples were added. The final benchmark contains 404 accepted tasks: 166 stock, 94 bond, 122 fund, and 22 macro tasks. Appendix B gives the curation log and reference-program verification.

Dataset composition and task taxonomy. BuySideFlow contains 404 practical research tasks across stock, fund, bond, and macro research. Stocks, funds, and bonds form the primary slices; the smaller macro slice adds time-series and release-calendar coverage. The tasks involve observation dates, report periods, disclosure status, investable universes, business definitions, rolling windows, and required output formats.

We classify tasks by executable workflow. Semantically, 312 tasks are SQL-centered, testing schema grounding, point-in-time filtering, joins, aggregation, window functions, ranking, and sorting. The remaining 92 tasks require Python-mediated analysis or artifact generation after retrieval, such as rolling time-series alignment, regression, optimization, attribution, visualization, or packaging. The full level taxonomy is reported in Appendix B.

Time anchoring. The central design constraint is point-in-time admissibility. A task may be anchored by a trading date, report date, month end, quarter end, year end, event window, or structured input. For financial statements, admissibility requires disclosure-date filtering before selecting the latest reporting period; for status-like tables, it requires effective-date intervals rather than current labels. The audit identifies 40 anchors from structured inputs, 290 from instruction text, 36 from derived date expressions, and 38 year-level anchors. Future realized returns, drawdowns, and post-event outcomes are treated as offline labels only.

Each reference solution is checked by a static time audit over SQL and Python. The audit reports no runtime date-function violations, no missing financial disclosure guards, and no missing status effective-window guards. Detailed audit rules and advisory checks are reported in Appendix C.

Schema context and output contracts. The controlled schema exposes 108 documented tables to the agent; the current reference programs exercise 78 of them. Unused tables are retained because they are part of the licensed schema and remain in the table-search space. Agents receive shared schema assets, a compact table catalog, business-background rules, table-selection guidance, and fund-specific rules; these assets contain no reference outputs.

Most tasks evaluate structured outputs: 392 CSV tasks, 5 mixed-artifact tasks, and 7 figure-oriented tasks. For tabular tasks, the output contract specifies column names, ordering, units, rounding, and row-level requirements when applicable. For Python-mediated tasks, the evaluator runs the submitted workflow and compares declared result variables or generated artifacts with the reference output. The mixed and figure-oriented tasks are reported as secondary stress tests rather than as a standalone multimodal benchmark.

4 Schema, Release Boundary, and Controlled Reproducibility

Schema and benchmark context. BuySideFlow is built on a licensed Chinese financial database rather than a benchmark-specific schema. The controlled snapshot exposes 108 documented tables covering securities, quotes, financial statements, index constituents, funds, bonds, credit ratings, yield curves, and macro indicators. The current references exercise 78 of these tables. We report 108 as the schema search space available to agents, not as a claim that every table is used by the current task set. Unused tables are retained because they are part of the production schema and remain distractors during table selection.

Agents receive schema information through controlled assets and tools rather than unrestricted database dumps. The assets include a full schema file for validation, a compact table catalog for search, and business-guidance files covering disclosure dates, join keys, fund conventions, classification vintages, manager tenure, and common formulas. They are shared by all baseline agents, included in provenance hashes, and contain no reference outputs or hidden labels. Non-sensitive schema summaries are included in the anonymized public audit package; licensing-restricted schema assets are available only in the controlled execution environment.

Release boundary. The public release is an audit package, not a redistribution of the licensed database. It includes task statements, structured inputs, output contracts, metadata, audit and evaluation code, prompt assets, table-usage summaries, benchmark reports, figures, and provenance records. It does not include database dumps, raw licensed rows, unrestricted query outputs, credentials, or proprietary source documents.

Reference programs and outputs follow the same boundary. Inside the authorized environment, each task has executable reference SQL or Python and expected artifacts. For public audit, we release hashes and output contracts; reference code or artifacts are released only when they do not expose licensed rows or restricted schema details. Full local execution therefore requires controlled database access, while the public package supports inspection, metadata validation, and review of the evaluation protocol.

Controlled reproducibility. Full evaluation is performed against a fixed database snapshot. A paper-grade run must set a non-empty `DB_SNAPSHOT_ID`; the evaluator records this identifier together with hashes for benchmark sources, schema assets, prompt assets, reference files, sidecars, evaluator code, and judge caches. This identifier records which database state was used to compute the reported scores.

External reproduction is supported in two ways. Authorized users may receive read-only, query-only access to the fixed managed snapshot under the relevant data-provider license. When direct access is not feasible, users may submit model outputs, trajectories, or workflows to an official evaluation service, which executes them against the same snapshot and returns scores, diagnostics, tool-use summaries, and provenance records.

BuySideFlow therefore provides three levels of reproducibility: public inspection of the benchmark definition, controlled execution for authorized users, and official evaluation against the maintained snapshot. We do not claim that every user can locally reproduce database query results without licensed data access. The claim is that the task set, evaluation protocol, database state used for scoring, and non-public execution artifacts are versioned and auditable.

5 Evaluation Protocol and Baselines

Submission format and execution. For each task, the model submits a JSON object in one of three modes: `sql`, `sql+python`, or `python`. In `sql` mode, the executable answer is a MySQL query. In `sql+python` and `python` modes, the submitted Python code may call the database connector, perform downstream computation, and return declared result variables or artifacts. Structured outputs must be DataFrames or numeric scalars; mixed outputs must produce the required table, text, image, or packaged artifact. The evaluator runs the submission and compares its outputs with reference artifacts; it does not grade code as standalone text. Execution uses MySQL 8.0 and Python with the benchmark database connector. Baseline agents receive identical schema assets, table catalog, business-background rules, table-selection guidance, and fund-specific rules. The interface mode is separate from the SQL-centered/Python-mediated task taxonomy: many SQL-centered tasks still run through Python because Python also serves as a database wrapper and output-normalization layer.

Leakage guards. Before execution, the evaluator applies static guards against runtime-clock leakage. SQL submissions are rejected if they contain `CURDATE()`, `CURRENT_DATE`, `NOW()`, `SYSDATE()`, or `CURRENT_TIMESTAMP`. Python submissions are rejected if they call `date.today()`, `datetime.now()`, `datetime.today()`, `pd.Timestamp.today()`, or `pd.Timestamp.now()`. The same guard applies to SQL strings passed through `query_to_dataframe(sql)` and related connector calls. Reference programs are checked separately by the time-audit script described in Section 3.

Scoring. The primary metric is tabular execution accuracy. For CSV tasks, the evaluator runs both reference and generated workflows, normalizes the resulting tables, and compares them under the task's output contract: output count, column names and order when required, row structure, dates, codes, text fields, numeric values, units, and rounding. For task i, the evaluator returns score s_i and maximum m_i. For standard CSV tasks, $m_i = 1$ and $s_i \in \{0, 1\}$; mixed and visual tasks may receive partial scores from a checklist evaluator. We report

$$\text{Score} = \frac{\sum_i s_i}{\sum_i m_i}, \qquad \text{FullScore} = \frac{1}{N} \sum_i \mathbb{1}[s_i = m_i].$$

The main metric is the primary tabular score over 392 CSV tasks. The remaining 12 mixed-or-visual tasks are reported as secondary metrics.

Diagnostics and tool use. Failed tasks receive a diagnostic type. Generation errors include syntax errors, execution errors, timeouts, schema mismatches, and missing outputs. Comparison errors include format mismatches, logic mismatches, and judge mismatches. We use `logic_mismatch` for executed workflows whose outputs disagree with the reference. Generated SQL and Python additionally receive non-exclusive `pit_diagnostic_flags` for runtime-date leakage, missing disclosure guards, current-state filters, missing classification vintages, or suspected use of future labels. These flags are advisory: they do not change scores and have not been manually validated as failure causes.

The agent receives the task statement, output contract, business metadata, and available-table list. It can use schema tools, a trading-calendar tool, an execution tool, and a submission tool: `search_tables`, `describe_tables`, `get_columns`, `search_columns`, `request_schema`, `get_trading_days`, `run_code`, and `submit`. Schema tools expose table descriptions and column definitions without revealing reference answers; `run_code` executes the candidate workflow for debugging. Each formal run exports a per-task tool-use table used to analyze schema exploration, execution loops, and premature submission.

Baselines. We evaluate seven controlled LLM agent runs under the same scaffold, prompt assets, schema tools, controlled database snapshot, output contracts, and evaluator. All runs are launched through `BuySideFlow/run_text2sql.py`. The setting is zero-shot at the task level. The suite includes six general production-oriented models and one coding-specialized variant. Including Qwen3-Coder-Plus tests whether stronger code generation alone closes the gap on point-in-time

Model	Access channel	Report date	Runner setting
GPT-5.5	Provider API	2026-04-28	default; temp/top-p not set
Claude-Opus-4.7	Provider API	2026-04-28	default; temp/top-p not set
Qwen3-Coder-Plus	Provider API	2026-04-29	temp=0.0, top-p=null
Qwen3.6-Plus	Provider API	2026-04-29	temp=0.0, top-p=null
DeepSeek-V4-Flash	Provider API	2026-04-30	temp=0.0, top-p=null
Kimi-K2.6	Provider API	2026-04-30	temp=0.60, top-p=0.95
GLM-5.1	Provider API	2026-05-01	temp=0.0, top-p=null

Table 3: Controlled baseline configurations. All runs were launched through `BuySideFlow/run_text2sql.py`. Report dates are parsed from the formal benchmark report filenames under `trajectories/`. Runner settings are the values recorded by the benchmark runner.

Run	FullScore	Overall Score	Primary CSV	Secondary
GPT-5.5	55/404	55.610/404	55.000/392	0.610/12
Claude-Opus-4.7	54/404	54.693/404	54.000/392	0.693/12
GLM-5.1	53/404	54.059/404	53.000/392	1.059/12
DeepSeek-V4-Flash	47/404	48.274/404	47.000/392	1.274/12
Kimi-k2.6	42/404	43.027/404	42.000/392	1.027/12
Qwen3.6-Plus	37/404	37.748/404	37.000/392	0.748/12
Qwen3-Coder-Plus	23/404	23.721/404	23.000/392	0.721/12

Table 4: Controlled baseline results. Primary CSV is the main metric; mixed and figure-oriented tasks are secondary.

financial workflows. Table 3 reports the model service accessed by the runner, the report date parsed from the formal benchmark report filename, and the runner setting recorded for that run. Runner settings should not be read as claims about provider-level decoding support when a provider does not expose the corresponding option. All reports share the same benchmark-source hash, schema and prompt-asset hashes, reference hash, sidecar hash, evaluator version, and database snapshot identifier.

6 Results, Error Analysis, and Discussion

Overall performance. Table 4 reports seven controlled runs under the same task set, prompt assets, schema tools, output contracts, evaluator, and database snapshot. The strongest run, GPT-5.5, solves 55 of 404 tasks. Claude-Opus-4.7 and GLM-5.1 are close, with 54 and 53 full-score tasks. The remaining runs solve between 23 and 47 tasks. These single controlled runs should not be read as a definitive ranking of model families; the stable conclusion is that all agents solve only a small fraction of BuySideFlow.

Workflow difficulty. Table 5 separates SQL-centered tasks from Python-mediated analytical or artifact workflows. The best SQL-centered result is GLM-5.1 with 49/312 full-score tasks. The best Python-mediated result is shared by GPT-5.5 and Claude-Opus-4.7, each solving 8/92 tasks. Qwen3-Coder-Plus solves only 2/92 Python-mediated tasks, suggesting that code generation alone does not address the harder part of the benchmark: selecting the admissible historical slice, applying the correct financial definition, and satisfying the output contract.

Failure modes. Most failures are executable workflows whose outputs disagree with the reference. Table 6 gives the per-run counts behind the aggregate number: across seven runs, `logic_mismatch` accounts for 2317 failed tasks. This is the dominant category for every run, including Qwen3-Coder-Plus, which has more generation and schema failures than the other models. The common failure pattern is therefore not inability to run code, but returning a table or artifact with the wrong table choice, join, filter, formula, temporal condition, or output contract.

Run	SQL Full	SQL Score	Py Full	Py Score
GPT-5.5	47/312	47.000/312	8/92	8.610/92
Claude-Opus-4.7	46/312	46.000/312	8/92	8.693/92
GLM-5.1	49/312	49.000/312	4/92	5.059/92
DeepSeek-V4-Flash	45/312	45.000/312	2/92	3.274/92
Kimi-k2.6	41/312	41.000/312	1/92	2.027/92
Qwen3.6-Plus	33/312	33.000/312	4/92	4.748/92
Qwen3-Coder-Plus	21/312	21.000/312	2/92	2.721/92

Table 5: Performance by workflow type. SQL denotes the 312 SQL-centered tasks; Py denotes the 92 Python-mediated tasks.

Run	Logic	Format	Gen./schema	Other
GPT-5.5	323	11	9	6
Claude-Opus-4.7	337	12	1	0
GLM-5.1	334	11	5	1
DeepSeek-V4-Flash	339	13	4	1
Kimi-k2.6	341	12	9	0
Qwen3.6-Plus	343	10	14	0
Qwen3-Coder-Plus	300	4	77	0

Table 6: Main failure categories by run. Logic denotes executed workflows whose outputs disagree with the reference.

Point-in-time diagnostics. Figure 1 summarizes PIT diagnostic flags on failed generated submissions. The flags are static, non-exclusive, and advisory; they do not change scores and have not been manually validated as failure causes.

The largest recurring signal is missing classification vintage. It appears 9 times for GPT-5.5, 35 for Claude-Opus-4.7, 44 for GLM-5.1, 44 for DeepSeek-V4-Flash, 48 for Kimi-k2.6, 65 for Qwen3.6-Plus, and 60 for Qwen3-Coder-Plus. Financial disclosure-guard signals are especially frequent for Kimi-k2.6 and Qwen3-Coder-Plus, with 33 and 65 flags. Runtime-date leakage is not observed among executed submissions because submissions containing blocked runtime date functions are rejected before execution. GPT-5.5 has the largest future-label-suspected count, with 11 flags; manual inspection shows that these cases often involve post-event return or drawdown computations whose role is ambiguous under static analysis. We therefore treat the flag as an audit prompt, not as evidence of a model-specific causal failure.

Tool-use behavior. Figure 2 summarizes process metrics from the per-task tool-use CSV files. Six runs execute candidate code on every task before submission; Qwen3-Coder-Plus has six tasks with no `run_code` call. The early-submit rate is 0.0% for six runs and 1.5% for Qwen3-Coder-Plus. Failures therefore do not mainly come from submitting without execution.

The more common pattern is exploration without convergence. GPT-5.5 has the lowest average `run_code` count, 3.45 calls per task, yet obtains the strongest overall score. Kimi-k2.6 has the highest average `run_code` count, 32.61 calls per task, but does not achieve a higher score. Navigation-trap rates range from 60.4% to 92.1%. Tool access is necessary for this benchmark, but tool-use volume is not a sufficient explanation of success.

Discussion. The results show a persistent gap between executable workflows and admissible research outputs. Current agents often inspect schema, run candidate code, and return an artifact, but still miss the table choice, temporal guard, financial definition, or output contract needed for a correct answer. This explains why the coding-specialized baseline does not dominate: stronger code generation helps only after the workflow has grounded the right historical slice and business rule.

The SQL-centered/Python-mediated split shows that post-retrieval analysis remains a major bottleneck. Python-mediated tasks require the model to preserve the point-in-time slice while computing rolling statistics, attribution, regression, visualization, or packaged outputs. Failures in these tasks often begin before Python computation, at the retrieval and temporal-filtering stage. BuySideFlow is

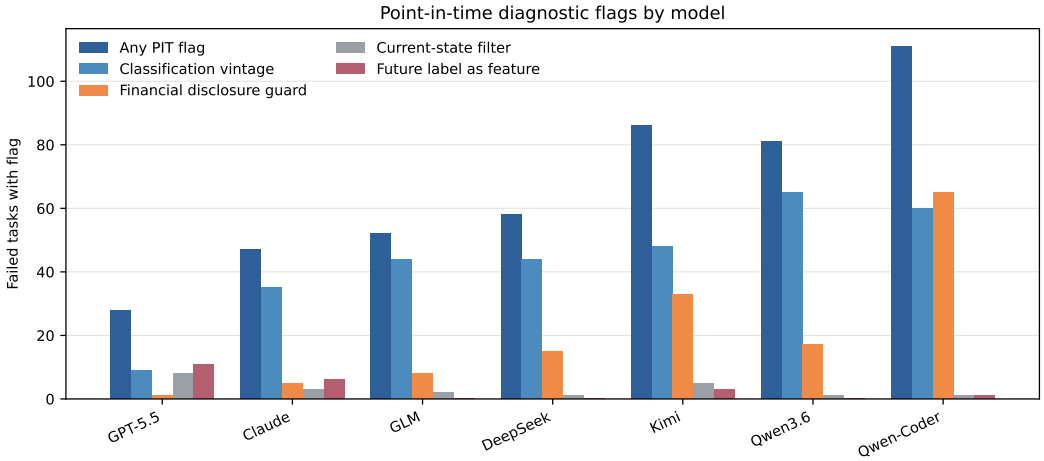

Figure 1: PIT diagnostic flags on failed tasks across seven controlled runs.

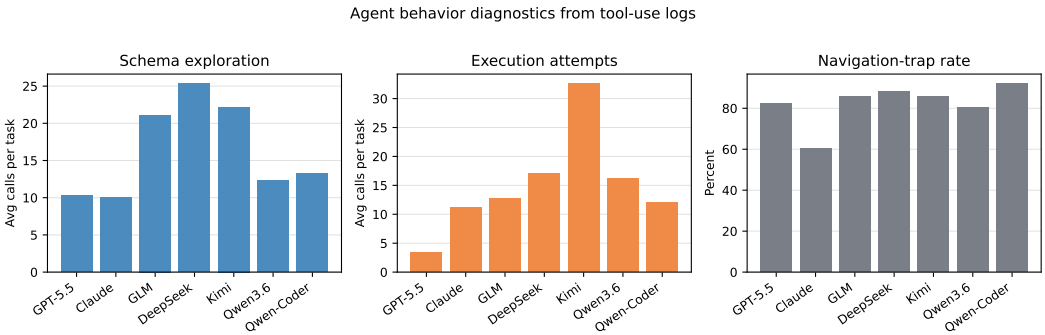

Figure 2: Agent behavior diagnostics from per-task tool-use logs. More schema exploration or execution attempts do not by themselves imply higher task success.

therefore better read as a controlled stress test for temporally admissible buy-side research workflows than as a leaderboard for small differences between model releases. A credible system in this setting should be evaluated by execution accuracy, temporal admissibility, output-contract compliance, provenance, and the process evidence left by schema and execution tools.

7 Limitations

BuySideFlow is less openly reproducible than public-database benchmarks because the underlying financial data are licensed. The public audit package exposes task text, metadata, code, reports, and provenance records, but not licensed rows, unrestricted query outputs, or credentials. Full execution requires authorized access to the fixed database snapshot or use of the official evaluation service.

The current benchmark is concentrated rather than web-scale. It contains 404 tasks, with stocks, funds, and bonds as the primary slices; the macro slice and the 12 mixed or figure-oriented tasks are smaller and should be read descriptively. Future versions should broaden these slices while preserving point-in-time auditability.

The baselines are single controlled runs over production model services. Provider implementations and model revisions may change, and not all providers expose identical decoding controls. The PIT diagnostic flags are static advisory checks: they locate leakage risks but are not manually validated failure causes. The reported scores should therefore be read as evidence that the setting is difficult, not as a final ranking of model families.

References

David H. Bailey, Jonathan M. Borwein, Marcos Lopez de Prado, and Qiji Jim Zhu. The probability of backtest overfitting. *Journal of Computational Finance*, 20(4):39–69, 2016.

Rolf W. Banz and William J. Breen. Sample-dependent results using accounting and market data: Some evidence. *The Journal of Finance*, 41(4):779–793, 1986.

Antoine Bigeard, Rayan Krishnan, Shirley Wu, and Langston Nashold. Finance agent benchmark: Benchmarking LLMs on real-world financial research tasks, 2025.

Stephen J. Brown, William Goetzmann, Roger G. Ibbotson, and Stephen A. Ross. Survivorship bias in performance studies. *The Review of Financial Studies*, 5(4):553–580, 1992.

Peter Baile Chen, Fabian Wenz, Yi Zhang, Devin Yang, Justin Choi, Nesime Tatbul, Michael Cafarella, Çağatay Demiralp, and Michael Stonebraker. BEAVER: An enterprise benchmark for text-to-SQL, 2024.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. FinQA: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711, 2021.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6279–6292, 2022.

Xinyi He, Mengyu Zhou, Xinrun Xu, Xiaojun Ma, Rui Ding, Lun Du, Yan Gao, Ran Jia, Xu Chen, Shi Han, Zejian Yuan, and Dongmei Zhang. Text2Analysis: A benchmark of table question answering with advanced data analysis and unclear queries. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18206–18213, 2024.

Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Jiwei Li, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. InfiAgent-DABench: Evaluating agents on data analysis tasks, 2024.

Nan Huo, Xiaohan Xu, Jinyang Li, Per Jacobsson, Shipei Lin, Bowen Qin, Binyuan Hui, Xiaolong Li, Ge Qu, Shuzheng Si, Linheng Han, Edward Alexander, Xintong Zhu, Rui Qin, Ruihan Yu, Yiyao Jin, Feige Zhou, Weihao Zhong, Yun Chen, Hongyu Liu, Chenhao Ma, Fatma Ozcan, Yannis Papakonstantinou, and Reynold Cheng. BIRD-INTERACT: Re-imagining text-to-SQL evaluation for large language models via lens of dynamic interactions. In *The Thirteenth International Conference on Learning Representations (ICLR), Oral Presentation*, 2026. URL `https://arxiv.org/abs/2510.05318`.

Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning*, pages 18319–18345, 2023.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. In *International Conference on Learning Representations*, 2025.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems*, 2023.

Marcos Lopez de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.

Qingkai Min, Yuefeng Shi, and Yue Zhang. A pilot study for Chinese SQL semantic parsing. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.

Varshini Reddy, Rik Koncel-Kedziorski, Viet Dac Lai, Michael Krumdick, Charles Lovering, and Chris Tanner. DocFinQA: A long-context financial reasoning dataset, 2024.

Avinash Kumar Singh, Bhaskarjit Sarmah, and Stefano Pasquali. FINCH: Financial intelligence using natural language for contextualized SQL handling, 2025.

Lijie Wang, Ao Zhang, Kun Wu, Ke Sun, Zhenghua Li, Hua Wu, Min Zhang, and Haifeng Wang. DuSQL: A large-scale and pragmatic Chinese text-to-SQL dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6923–6935, 2020.

Fabian Wenz, Omar Bouattour, Devin Yang, Justin Choi, Cecil Gregg, Nesime Tatbul, and Çağatay Demiralp. BenchPress: A human-in-the-loop annotation system for rapid text-to-SQL benchmark curation. In *Conference on Innovative Data Systems Research (CIDR)*, 2026.

Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, et al. FinBen: A holistic financial benchmark for large language models. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, 2018.

Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter S. Lasecki, and Dragomir Radev. CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019a.

Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. SParC: Cross-domain semantic parsing in context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4511–4523, 2019b.

Lingfeng Zeng, Fangqi Lou, Zixuan Wang, Jiajie Xu, Jinyi Niu, Mengping Li, Yifan Dong, Qi Qi, Wei Zhang, Ziwei Yang, Jun Han, Ruilun Feng, Ruiqi Hu, Lejie Zhang, Zhengbo Feng, Yicheng Ren, Xin Guo, Zhaowei Liu, Dongpo Cheng, Weige Cai, and Liwen Zhang. FinGAIA: A Chinese benchmark for AI agents in real-world financial domain, 2025.

Chao Zhang, Yuren Mao, Yijiang Fan, Yu Mi, Yunjun Gao, Lu Chen, Dongfang Lou, and Jinshu Lin. FinSQL: Model-agnostic LLMs-based text-to-SQL framework for financial analysis, 2024.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3277–3287, 2021.

A Dataset Card and Verifiable Statistics

The anonymized public audit package includes the dataset card, Croissant metadata, task files, generated manifests, schema summaries where licensing permits, audit and evaluation code, prompt assets, benchmark reports, tool-use logs, figures, and controlled-access documentation. The licensed database rows, credentials, unrestricted query outputs, and proprietary source documents are not redistributed. The repository URL is omitted during double-blind review and will be released after acceptance.

Dataset card and Croissant metadata. The dataset card describes task construction, temporal anchoring, leakage guards, output contracts, evaluation metrics, provenance, release boundaries, and known limitations. The Croissant JSON-LD file, `croissant_metadata.json`, records the dataset identifier, version, language, public file distribution, license and access boundary, responsible-use notes, and controlled-execution limitations. Its purpose is to make the public release machine-readable without implying that the licensed database itself is open data.

Verifiable statistics. Table 7 reports counts derived from the public task files, generated manifest, and table-usage records. These statistics can be checked without database credentials.

Statistic	Value
Total tasks	404
Stock / fund / bond / macro	166 / 122 / 94 / 22
SQL-centered / Python-mediated workflows	312 / 92
L1 / L2 / L3 / L4 tasks	19 / 261 / 112 / 12
CSV / mixed / figure-oriented tasks	392 / 5 / 7
Tasks with strict output schema	404
Time anchors: input / instruction / derived / year-level	40 / 290 / 36 / 38
Future-label exceptions	13
Documented schema tables / reference-exercised tables	108 / 78
Reference programs with `refer.py` / `refer.sql`	404 / 312
Tasks with `result.csv` / figure artifacts	404 / 7

Table 7: BuySideFlow statistics derived from public manifest and task artifacts.

Verification boundary. The public package supports inspection of task text, metadata, taxonomy, output contracts, prompt assets, schema summaries where licensing permits, audit rules, evaluator code, reported results, tool-use logs, and provenance records. It does not allow unauthenticated users to rerun database queries locally or reconstruct licensed database rows. Full-score reproduction requires the fixed controlled database snapshot, either through authorized read-only access or through the official evaluation service. Every paper-grade run records `DB_SNAPSHOT_ID` and SHA256 hashes for benchmark sources, schema assets, prompt assets, reference files, sidecars, evaluator code, and judge caches.

B Task Taxonomy, Examples, and Review Log

Task taxonomy. BuySideFlow uses two task views. The level taxonomy describes task complexity; the workflow taxonomy separates SQL-centered retrieval from Python-mediated analysis after retrieval. Python is also used as a database wrapper, so implementation file type alone is not a reliable task label.

Representative examples. Table 9 gives paraphrased examples of common task patterns.

Curation log. Task construction started from 411 draft tasks and ended with 404 accepted tasks. Review changed the pool through both deletion and addition. Table 10 summarizes the main domain-level changes.

Reference-program verification. Reference programs were checked in three manual passes. The rates in Table 11 denote the share of task references accepted in that pass without further correction,

Level	Description	SQL-centered	Python-mediated	Total
L1	Direct retrieval	19	0	19
L2	SQL filtering, joining, aggregation, and ranking	261	0	261
L3	SQL–Python analytical computation	31	81	112
L4	Mixed or figure-oriented artifact generation	1	11	12
Total		312	92	404

Table 8: Task levels and workflow types.

Type	Domain	Typical request	Main challenge
L1 retrieval	Macro	Retrieve a dated macro indicator series	Date and period alignment
L2 SQL-centered	Stock	Screen securities as of a research date and return a ranked list	PIT universe, joins, filters, ranking
L2 SQL-centered	Bond	Select bonds under valuation, rating, maturity, or event constraints	Validity windows, code deduplication
L3 SQL–Python	Fund	Compute rolling return, drawdown, or attribution after SQL retrieval	Time-series alignment and formulas
L3 SQL–Python	Stock	Run a factor test or event-window analysis	PIT sample construction and post-retrieval computation
L4 mixed/figure	Fund/stock	Produce a table plus figure or packaged artifact	Artifact generation and output contract

Table 9: Representative BuySideFlow task patterns.

not an estimate of residual error in the released benchmark. The released references are the versions accepted after the final pass.

Because each benchmark label is an executable workflow, the review process is closer to reference construction and adjudication than to majority-vote annotation.

C Time-Audit, Output Contracts, and Evaluation Details

Time-audit rules. Table 12 summarizes the static checks used by the reference audit and generated-code diagnostics. The rules flag common leakage channels; ambiguous cases are left visible for manual review.

The current reference audit reports no runtime date-function violations, no missing financial disclosure guards, and no missing status effective-window guards. It also reports `cs_riskalert_publish_guard_missing=62`, an advisory check on whether risk-alert announcement or update timestamps are bounded by the observation date. This advisory is reported separately because many risk-alert references define state through implementation and removal dates.

Future labels. Thirteen tasks allow future realized returns, drawdowns, redemption windows, or post-event outcomes. These values are used only as offline labels or ex-post evaluation targets, not as features, filters, or point-in-time inputs. The exceptions are recorded in `manifest_overrides.json`.

Output contracts and errors. For CSV tasks, the evaluator checks output count, column names, column order when required, row structure, dates, codes, text fields, numeric values, units, rounding, and sorting rules. Mixed and figure-oriented tasks are reported as secondary metrics. Tables inside mixed outputs are still evaluated by the structured comparator; text or image components use a checklist-based judge, with judge-cache hashes recorded in the report.

The evaluator records syntax errors, execution errors, schema mismatches, timeouts, missing outputs, format mismatches, logic mismatches, and judge mismatches. Executed workflows whose outputs disagree with the reference keep `logic_mismatch` as the main category. Generated code may also receive non-exclusive PIT di-

Domain	Draft issue	Curation action	Final effect
Stock	Too many cross-sectional screens and ranked lists	Removed near-duplicates; revised 16 tasks; added event, factor-test, and portfolio tasks	171 draft tasks to 166 accepted tasks
Bond	Many observation-date screens, sorting tasks, and single-event replays	Removed high-similarity tasks; added validity, interpolation, deduplication, event-window, and fallback-rule cases	98 draft tasks to 94 accepted tasks
Fund	Some tasks lacked explicit TopK, output fields, candidate sets, ETF keywords, or thresholds	Standardized 14 tasks and aligned wording with reference outputs	122 draft tasks to 122 accepted tasks
Macro	Macro slice was small and lacked PMI examples	Added PMI-related tasks and corrected indicator definitions	20 draft tasks to 22 accepted tasks

Table 10: Domain-level curation actions from task review.

Pass	Domain	Tasks	Accepted	Main issues found
1	Macro	22	70%	Time-series logic and period alignment
1	Stock	166	45%	Backtesting logic, factor construction, multi-table joins
1	Fund	122	52%	Workflow logic, specialized algorithms, process tracking
1	Bond	94	50%	Curve interpolation and percentile calculation
2	Macro	22	100%	No further corrections
2	Stock	166	85%	Regression-style tasks and factor construction
2	Fund	122	92%	Holdings analysis, turnover statistics, correlation logic, recommendation logic
2	Bond	94	90%	Interpolation, percentile calculation, bond-code handling
3	All	404	100%	Final accepted references

Table 11: Manual verification of reference programs. Acceptance means that the reference program passed that review round without further correction.

agnostic flags: `runtime_date_leakage`, `financial_publish_guard_missing`, `current_state_status_filter`, `classification_vintage_missing`, and `future_label_as_feature_suspected`. The flags support error analysis and do not change scores.

D Schema, Controlled Access, Provenance, and Additional Results

Schema and access. The controlled snapshot exposes 108 documented tables; the current references exercise 78. The schema covers securities, market quotes, financial statements, index constituents, funds, bonds, yield curves, credit ratings, and macro indicators. Schema summaries are public where licensing permits; restricted schema assets are available only in the controlled environment and are identified by hashes. Licensed database rows are not redistributed. Full execution uses either read-only access to the fixed database snapshot or the official evaluation service.

Provenance. Each paper-grade run sets `DB_SNAPSHOT_ID`. Reports record SHA256 hashes for benchmark sources, schema assets, prompt assets, reference files, sidecars, evaluator code, and judge caches. The generated manifest stores per-task metadata and hashes, but the evaluator remains the source of truth.

Additional results. Tables 13 and 14 give additional PIT diagnostics and process metrics. The complete score table is in Section 6 and is not repeated here.

E Prompts, Tool Definitions, and Public Result Artifacts

Prompt assets. All baseline agents receive the same prompt assets. `business_background.txt` records general financial-database conventions and point-in-

Audit target	Examples	Purpose
Runtime SQL date functions	Block `CURDATE()`, `CURRENT_DATE`, `NOW()`, `SYSDATE()`, `CURRENT_TIMESTAMP`	Prevent wall-clock leakage
Runtime Python date functions	Block `date.today()`, `datetime.now()`, `datetime.today()`, `pd.Timestamp.today()`, `pd.Timestamp.now()`	Prevent hidden runtime-date leakage
SQL inside Python connector calls	Apply SQL lint to strings passed to `query_to_dataframe(sql)`	Prevent hiding date functions in Python strings
Financial disclosure guards	Accept `INFOPUBLDATE <= anchor`, `DATE(INFOPUBLDATE) <= anchor`, bounded `BETWEEN`	Ensure statements were disclosed by the research date
Status-window guards	Accept intervals over `EFFECTIVEDATE`, `CANCELDATE`, `LISTEDDATE`, `IMPLEMENTDATE`, `REMOVEDATE`	Avoid current-state entity leakage
Derived date guards	Accept `DATE(field) <= anchor`, `field < DATE_ADD(anchor, INTERVAL 1 DAY)`, listing-age filters	Reduce reference-audit false positives

Table 12: Time-audit and leakage-guard patterns.

Run	Any	Class.	Disclosure	Current	Future
GPT-5.5	28	9	1	8	11
Claude-Opus-4.7	47	35	5	3	6
GLM-5.1	52	44	8	2	0
DeepSeek-V4-Flash	58	44	15	1	0
Kimi-k2.6	86	48	33	5	3
Qwen3.6-Plus	81	65	17	1	0
Qwen3-Coder-Plus	111	60	65	1	1

Table 13: PIT diagnostic flags on failed generated submissions. Class. denotes missing classification vintage. Flags are non-exclusive and used for error analysis rather than scoring.

time rules. `fund_business_rules.txt` records fund-domain conventions, including share-class filtering, fund classification, adjusted NAV, manager tenure, and holdings look-through. `selection_guidance.txt` provides table- and field-selection guidance. These files contain no reference outputs.

Tool definitions. The agent can call schema tools, a trading-calendar tool, an execution tool, and a submission tool: `search_tables`, `describe_tables`, `get_columns`, `search_columns`, `request_schema`, `get_trading_days`, `run_code`, and `submit`. Schema tools expose descriptions and column definitions. `run_code` executes the candidate workflow against the controlled database snapshot. The main setting does not expose reference outputs.

Public result artifacts. The public package includes benchmark reports and per-task tool-use CSV files for the controlled runs. The reports contain scores, error categories, PIT summaries, and provenance hashes. The tool-use CSV files contain one row per task and are used to compute the process metrics reported in Section 6 and Table 14. Full trajectories are released only after removing credentials, restricted rows, and non-public schema details.

Run	Avg. schema	Avg. run	NTR	ESR
GPT-5.5	10.33	3.45	82.4%	0.0%
Claude-Opus-4.7	10.02	11.14	60.4%	0.0%
GLM-5.1	21.13	12.78	85.6%	0.0%
DeepSeek-V4-Flash	25.35	17.08	88.1%	0.0%
Kimi-k2.6	22.19	32.61	85.9%	0.0%
Qwen3.6-Plus	12.30	16.14	80.2%	0.0%
Qwen3-Coder-Plus	13.29	12.07	92.1%	1.5%

Table 14: Process metrics from per-task tool-use logs. NTR is navigation-trap rate; ESR is early-submit rate.